OSSP HACKATHON PROJECT REPORT

PipeTalk

Inter Process Chat Application Using IPC

Two way communication between independent Linux processes using named pipes

Team 8
Department of Computer Science and Engineering

Team member	Roll number
Abhay Tripathi	2520030353
K. Pranav	2520030477
Jashwanth	2520030236

Guided by: Ms. Soumya

Hackathon date: 17 October 2026

This report describes the implemented Linux IPC prototype, its web interface, execution procedure and verification results. It distinguishes the C peer communication layer from the Python web bridge and documents the current scope of the system.

Abstract and Project Overview

PipeTalk is a local web application for demonstrating inter-process communication on Linux. Two independent C programs, identified as Process A and Process B, exchange messages through two named pipes. A Python server manages their lifecycle and connects browser input to the processes. The website presents the conversation, receiving confirmations and live process information in one workspace.

The project addresses the difficulty of observing IPC from a terminal-only example. Its browser interface lets evaluators submit a message while the Ubuntu terminal independently verifies the participating processes and FIFO files. A message is shown as delivered only after the destination C process reports that it read the corresponding frame. The browser therefore exposes an operating-system operation that can also be inspected outside the interface.

The implementation uses C, the Python standard library and HTML, CSS and JavaScript. It requires neither a database nor npm or pip packages. Ubuntu can run it natively, and Windows can run the backend through Ubuntu on WSL. The application listens on the local loopback interface and is intended for a controlled classroom demonstration.

Implemented outcome

The prototype supports bidirectional message transfer, UTF-8 text, multiline messages, process inspection, manual peer restart and JSON session export. Ten automated checks pass in the Linux build environment. The user-provided execution evidence also shows the application running on Ubuntu through WSL with two distinct peer PIDs. Full Windows launcher execution and browser interactions are not covered by the Linux automated suite.

Report organisation

Page	Content
3	Problem statement, objectives and requirements
4	IPC background and system architecture
5	Linux peer implementation and message framing
6	Python bridge, web interface and HTTP operations
7	Execution procedure and demonstration
8	Testing and verification results
9	Implementation screenshots and observations
10	Conclusion, limitations and references

Problem Statement and Requirements

Assigned problem

Batch 8: Inter-Process Chat Application Using IPC. Develop a two-way communication application where independent processes exchange messages using IPC mechanisms. The required outcome is a working prototype in which the communication is performed by operating-system processes rather than represented only by browser state.

Project objectives

The primary objective is to make bidirectional Linux IPC observable. Each endpoint must exist as an independent C process with its own PID. Messages must travel through identifiable Linux IPC channels. The interface must expose delivery evidence, while terminal commands must confirm the process and FIFO resources used by the backend.

The project also aims to simplify live execution through launch scripts, preserve text accurately, handle peer failure clearly and release operating-system resources when the server stops. These requirements support a repeatable demonstration without adding external services unrelated to the assigned problem.

Requirement	Implemented behaviour
Independent endpoints	Two instances of the compiled pipetalk_peer C program
Two-way communication	One named FIFO in each direction
Usable interface	Two chat panels with send controls and receiving status
Linux evidence	PIDs, /proc details, FIFO paths and transport events
Failure handling	Offline state, blocked sends and manual restart
Execution support	Ubuntu shell launcher and Windows WSL entry point
Session output	JSON download containing retained messages and events

Environment and constraints

The supported environment is Ubuntu 22.04 or newer with Python 3.10 or newer, GCC and a modern browser. Initial installation of missing compiler or Python prerequisites requires a network connection and suitable package-manager privileges. Normal operation is local and can run offline after prerequisites are installed.

The current version contains exactly two chat endpoints. Browser tabs connected to the same server share that session. Multi-user authentication, communication between separate computers and permanent message storage are outside the implemented scope.

IPC Background and System Architecture

Linux separates processes into independent execution contexts. IPC mechanisms provide controlled paths for exchanging data between them. A FIFO is a named pipe accessible through a filesystem path. The path identifies the channel, while pipe I/O is handled by the kernel rather than stored as an ordinary document [1].

Mechanism	Relevance to this project
Anonymous pipes	Used for bridge-to-peer stdin and peer-to-bridge stdout control streams
Named pipes or FIFOs	Selected for the actual A-to-B and B-to-A message transport
Shared memory	Alternative requiring additional coordination of shared data
Unix domain sockets	Alternative for local communication with a different connection model

Selected architecture

The browser submits HTTP requests to the Python bridge. The bridge creates FIFOs, starts both peers and forwards JSON frames through the selected peer stdin. That C peer writes its outgoing FIFO. The destination peer reads the frame and reports receipt through stdout. Bridge reader threads collect these events and update the session.

Component	Responsibility	Interface
Web browser	Message input and live presentation	HTTP JSON requests
Python bridge	Process lifecycle and event correlation	Peer stdin and stdout
Process A	Send to B and receive from B	a_to_b.fifo / b_to_a.fifo
Process B	Send to A and receive from A	b_to_a.fifo / a_to_b.fifo
Linux kernel	FIFO buffering and descriptor I/O	open, poll, write and read

Transport and control separation

The two named FIFOs carry the peer-to-peer message payloads. Anonymous pipes connect each peer to the bridge for commands and confirmation events. HTTP connects the browser to the bridge. These paths have different responsibilities and should be explained separately during evaluation. Browser polling refreshes the displayed session every 500 ms.

Each server run creates a private temporary directory. Process inspector exposes its actual FIFO paths and the current peer PIDs.

Linux Backend Implementation

The file **backend/peer.c** implements a peer program. The bridge launches the same executable twice with different incoming and outgoing FIFO arguments. Each instance has a separate PID and address space. The C program performs the FIFO read and write operations that satisfy the assigned IPC requirement.

Operation	Purpose in the implementation
os.mkfifo(path, 0o600)	The Python bridge creates each FIFO with owner-only read/write permissions
open()	Each C peer opens incoming and outgoing FIFO descriptors
poll()	Waits for stdin commands or incoming FIFO data
write()	Moves a submitted frame into the outgoing FIFO
read()	Obtains data from the incoming FIFO or control stdin
close()	Releases descriptors when the peer exits
sigaction() and SIGTERM	Allow peer termination and orderly restart

Opening channels and waiting for input

Each peer uses the Linux-specific FIFO opening mode **O_RDWR | O_NONBLOCK**. Opening a FIFO for reading and writing succeeds on Linux even when the other endpoint has not yet opened it [1]. This avoids startup ordering deadlocks. Each peer polls only its designated incoming FIFO and its stdin control stream. It does not read back its own outgoing channel.

Framing and atomic writes

Messages are encoded as newline-delimited JSON. Newlines inside a message are escaped by the JSON serializer, allowing the peer to detect frame boundaries. The C program keeps incomplete data in a bounded buffer until a full frame is available. Each outgoing write is checked against the descriptor PIPE_BUF limit. Linux provides atomic pipe writes up to PIPE_BUF, which is 4096 bytes on Linux [2].

The bridge limits user text to 1600 UTF-8 bytes and rejects a serialized frame larger than 4096 bytes. These are different checks because JSON escaping can increase the frame size. A full channel produces an explicit failure rather than a successful delivery claim.

Lifecycle and resource cleanup

The bridge sends SIGTERM to active peers during restart or shutdown, waits for them, and uses a kill fallback if termination does not complete. It then closes streams and removes temporary resources. Normal Ctrl+C shutdown cleans up both peers. A forced SIGKILL or system crash may leave the previous temporary directory behind.

Web Bridge and Interface Implementation

The file **backend/server.py** uses ThreadingHTTPServer and subprocess.Popen [3, 4]. Startup compiles the C source, creates FIFOs and launches both peers. One thread collects stdout from each peer. A lock protects shared session state.

HTTP operation	Purpose	Result
GET /api/state	Read processes, FIFO paths, messages and events	Current session JSON
POST /api/send	Submit sender A or B and text	202 with a pending message ID
POST /api/restart	Recreate both peers and channels	200 after successful startup
GET /api/export	Download retained messages and transport events	Session JSON attachment

Receiving confirmation

The bridge assigns each submission a unique identifier and records the selected sender. The receiving event must come from the expected destination peer, contain the expected identifier and preserve the submitted sender and text. Only then does the session record become delivered. Queueing a request or writing to the sending process stdin does not by itself establish delivery.

Interface modules

frontend/index.html defines the Live chat, Process inspector and How it works views. **frontend/style.css** provides the responsive layout. **frontend/app.js** submits messages, polls the state endpoint and updates the interface. Messages are inserted with textContent so text resembling HTML remains text rather than browser markup.

Process inspector displays PID, state, resident memory and descriptor data from Linux /proc, with commands for the actual runtime paths. Transport activity lists queue, write, read and lifecycle events. Scheduling can cause the bridge to observe a receive event before the corresponding sent event.

Input protection and retained state

The HTTP server binds to 127.0.0.1, checks the Host header and requires a session token for modifying requests. Foreign origins are rejected. User messages are never evaluated by a shell. These controls support the local demonstration scope and do not provide public multi-user authentication.

Memory retains 500 messages and 200 events. Counters cover the bridge run. Observed delivery time includes bridge scheduling and receiving confirmation. It is not a kernel-only or network latency benchmark.

Execution and Live Demonstration

Startup procedure

Extract the ZIP completely. On Windows with Ubuntu initialised in WSL, launch **START_WINDOWS.bat**. It converts the project path and directly executes the Ubuntu startup script. Missing Python or GCC prerequisites are installed through the setup script.

For a native Ubuntu terminal, change into the extracted project directory and run:

```
bash START_UBUNTU.sh
```

The default dashboard address is http://localhost:8080. If the port is occupied, startup selects another available port and prints the correct URL. The launcher terminal must remain open while the demonstration runs. Use a second Ubuntu terminal for inspection commands.

Demonstration sequence

#	Action	Evidence to show
1	Open Live chat and send A to B	Incoming message and delivered confirmation
2	Reply from B to A	Reverse direction through the other FIFO
3	Open Process inspector	Two different peer PIDs
4	Run the process and FIFO commands	Actual C executables and p file-type marker
5	Terminate only the displayed B PID	Offline state and disabled sending
6	Select Restart processes	New PIDs and restored messaging
7	Export the session	JSON containing retained events and messages

Ubuntu inspection commands

```
ps -eo pid,ppid,stat,comm,args | grep '[p]ipetalk_peer'
```

Copy the exact FIFO inspection command from Process inspector because its temporary directory changes across sessions. Alternatively, the supplied helper prints the current paths and suitable commands:

```
bash scripts/inspect.sh
```

For another port, use **bash scripts/inspect.sh 8090** with the actual port. Demonstrate failure using **kill -TERM** and the current B PID. Press Ctrl+C only after the demonstration. Do not use cat on an active FIFO because it can consume a peer message.

Testing and Verification Results

The packaged test suite was executed against the current implementation in the Linux build environment. It compiled the C source and launched actual peer processes. All ten unittest checks passed. The suite provides functional evidence for the local prototype rather than a performance or large-scale reliability claim.

Check	Behaviour verified	Result
Independent peers and FIFOs	Distinct PIDs, live peers, FIFO file types and resident memory	Pass
Bidirectional transport	A to B and B to A receiving confirmations	Pass
Text preservation	UTF-8, multiline text, quotes and HTML-like content	Pass
Validation and token checks	Invalid senders, empty or excessive text and missing token	Pass
Concurrent submissions	20 alternating submissions through six client threads	Pass
Static files and export	Dashboard delivery and exported message content	Pass
Peer failure and restart	Terminated peer is reported and communication recovers	Pass
Launcher lifecycle	Path with spaces, busy port fallback and runtime cleanup	Pass
WSL invocation contract	Linux command calls use direct WSL execution	Pass
Native path transport	Windows path arguments retain backslashes and spaces	Pass

Reproducing the checks

```
python3 -m unittest discover -s tests -v
make check
```

Additional checks completed successfully: strict GCC C11 compilation with -Wall -Wextra -Werror, Python syntax compilation, shell syntax checks and JavaScript syntax validation. The archive was checked for file integrity and required source files.

Interpretation of results

The tests confirm the exercised functional paths. WSL checks on Linux do not execute PowerShell or Windows. Separate execution screenshots show the Ubuntu/WSL server and two C peers. Browser interactions are not automated. No throughput or latency benchmark is claimed.

Implementation Evidence

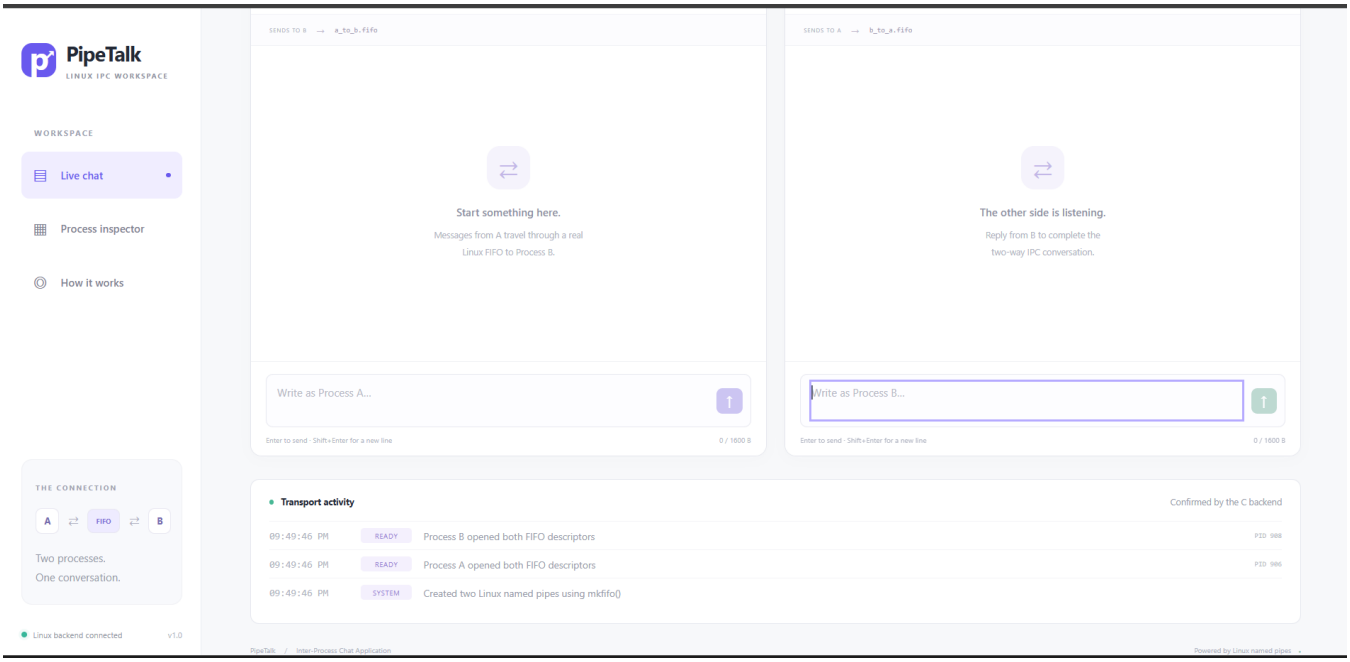


Figure 1. Actual PipeTalk interface in the initial connected state. The transport log shows both peers ready and two named pipes created. No message-delivery result is inferred from this empty conversation view.



Figure 2. Actual Ubuntu process listing supplied during execution. Process A has PID 1270 and Process B has PID 1272. Both show parent PID 1261, corresponding to the bridge for this session.

The process arguments show opposite FIFO assignments for A and B under the same temporary runtime directory. These PIDs and paths are session-specific observations, not fixed configuration values. State S indicates sleeping while waiting for input, and the plus marker identifies membership in the foreground process group. Live message delivery is verified by the receiving-process events and the integration tests described on page 8.

Conclusion and References

Conclusion

PipeTalk implements the assigned two-way IPC problem using two independent Linux C processes and two named FIFOs. The web interface provides message entry and observable delivery status, while terminal inspection exposes the actual operating-system resources. The separation between HTTP presentation, bridge control streams and peer-to-peer FIFO transport makes the implementation suitable for explaining Linux IPC during a live evaluation.

The implemented prototype supports process inspection, explicit peer restart and session export. Verification confirms the exercised local communication and lifecycle paths. The application also makes its limits visible, including in-memory retention, a two-endpoint model and local-only access.

Limitations and future work

Current history is lost when the server stops. The interface is not an authenticated messaging service and does not support separate remote computers. Future work could add durable storage, multiple named sessions and automated browser tests. A comparison with Unix domain sockets or shared memory would require a defined workload and new measurements before making performance claims. These extensions are proposed and are not part of this version.

References

[1] Linux man-pages project. fifo(7), FIFO special files.
https://www.man7.org/linux/man-pages/man7/fifo.7.html

[2] Linux man-pages project. pipe(7), overview of pipes and FIFOs.
https://www.man7.org/linux/man-pages/man7/pipe.7.html

[3] Python Software Foundation. subprocess, process management.
https://docs.python.org/3.11/library/subprocess.html

[4] Python Software Foundation. http.server, HTTP servers.
https://docs.python.org/3.12/library/http.server.html

Implementation sources: backend/peer.c, backend/server.py, frontend/app.js, launcher scripts, README.md and the packaged unittest suite. Figures 1 and 2 use actual execution screenshots supplied for this project. Technical web references were consulted on 16 October 2026.